Introduction to HTML and CSS

→ HTML (Hyper text Markup Language) is the standard markup language used to create and structure content on webpages. It defines the base structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML document are interpreted by web browsers to display the content in a structured & semantic organization of a webpage.

→ CSS (cascading style sheets) is a style sheet language used to control the presentation and visual appearance of HTML document. It is used to define properties such as colors, fonts, backgrounds spacing, borders, alignment, dimensions & layouts. CSS also supports responsive design, allowing webpage to adapt to different screen sizes and devices.

→ HTML & CSS work together to develop webpages where HTML provides the structure and content while CSS provides the styling,

layout, visual representation. The combin
-tion of both forms the foundation of
modern web development and enables the
creation of organized, attractive and
user-friendly web interfaces.

Difference between MERN stack and
FULL Stack

→ MERN stack is a Javascript based techno
gy stack used for developing modern web
applications. MERN stands for Mongo DB,
Express.js, React.js, Node.js. Mongo DB
is used for database management, Expres
.js handles the backend framework & API
React.js is used to develop frontend user
interface. & Node.js provides the server side
runtime environment

→ Full stack development refers the develo
-ment of both the frontend & backend
of a web application along with databa
integration. A fullstack developer works
with technologies used for designing the
user interface, implementing server-side
logic managing databases.

8/1

HTML components :-

1. HTML Tags - Define the structure and elements of a webpage.
2. Headings - used to display titles & headings using <h1> to <h6>
3. paragraphs - used to display text using <p> tag
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <OL> <UL>
7. Tables - used to organize data in rows & col
8. forms - used to collect user i/p using elements such as text boxes, buttons & check boxes
9. Div and span - used to group & organize webpage content.
10. Semantic elements - elements such as <header> <nav> <section>, <article>, <footer> provide meaningful webpage structure.

CSS components :-

1. Selectors - select HTML elements to apply styles
2. Properties - Define the visual characteristics of elements.
3. values - specify the setting of a CSS property
4. colors - define text, background, border colors
5. Fonts - control font family, size, style & weight
6. Backgrounds - set background colours & images
7. Borders - Define the border style, width & radius
8. margin - controls the space outside an element
9. padding - controls the space inside element
10. Layout - controls the arrangement of elements using flexbox, grid & positioning

HTML vs HTML5

HTML	HTML5
older version of HTML	latest major version of HTML
limited multimedia support	NO external plugs needed
uses external plugins for multimedia	supports audio & video
Has fewer, semantic tags	provides semantic tags
NO <canvas> element	supports <canvas>
complex Doctype declaration	supports local storage

CSS vs CSS3

CSS	CSS3
earlier version of CSS	Advanced version of CSS
Basic styling features	provides advanced styling features
limited animations	supports animations & transitions
NO flexible layout systems	supports flexbox & grid
limited visual effects	supports media queries

15/7

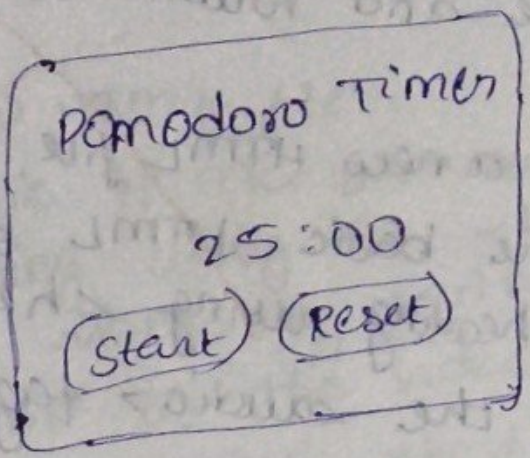
Design a static web page using HTML & CSS

Pomodoro Timmer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title Pomodoro Timer </title>
    <style>
        body {
            font-family: Arial;
            background: #f5f5f5;
            text allign : center;
            Padding : 50px;
        }
    .timer {
            width : 300px;
            margin : auto;
            padding : 30px;
            background : white;
            border-radius: 15px;
            box-shadow : 0 0 10px grey;
    }
    h1 {
        color : #e74c3c;
    }
    .time {
        font-size : 50px;
        margin : 20px;
    }
```

```
button{
    padding: 10px 20px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    background: #e74c3c;
    color: white;
}
</style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro Timer </h1>
        <div class = "time">2500 </div>
        <button> Start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

Olp :-



22/7

1) Design an HTML code using audio tags

code>

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My Favorite Song </h2>
    <audio controls>
        <source src = "song.mp3" type =
                                "audio/mpeg">
        Your browser doesn't support the audio tag.
    </audio>
</body>
</html>
```

o/p: Display an audio player with play, Pause and volume controls

Steps:

Create a new HTML file

Add the basic HTML structure

Add heading using <h2> tag

Insert the <audio> tag with control attribute

specify the audio file using <source> tag

Save and open the html file in webbrowser

(i) Design a HTML code using video tags

code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My Video </h2>
    <video width = "400" controls>
    <source src = "video.mp4" type = "video/
                                mp4">
    </video>
</body>
</html>
```

o/p:- Displays a video player with play, Pause, volume, and Full screen controls

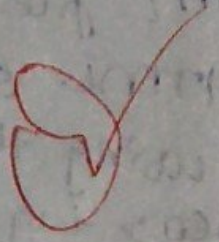
steps:

create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attribute specify the video file using <source> tag save the file and open it in web browser play the video using displayed controls

write a program using Javascript (arrays, functions)

code:

```
let number = [12,45, 8, 23, 56, 19, 34, 67, 90,];
function displayArray (arr) {
    console.log ("Array elements: ");
    for (let i=0; i<arr.length; i++) {
        console.log (arr[i]);
    }
}
function findSum (arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```



Experiment-4

Develop dynamic webpage using javascript

Program

```
<!DOCTYPE html>
<html>
<head>
<title> Dynamic webpage </title>
<style>
body {
    text-allign : center;
    font-family : Arial;
    margin-top : 100px;
}
button {
    padding : 10px 200px;
    font-size : 16px;
    cursor : pointer
}
</style>
</head>
<body>
<h1 id = "heading"> welcome to my
                    webpage </h1>
<p id = "message "> click the button to
                    change this webpage </p>
<button onclick ;"change page ()" >
                    click me </button>
```

```
<script>
    function changepage() {
    document.getElementById("heading")
      innerHTML = "welcome to javascript";
    document.getElementById("message")
    inner.HTML = "The webpage has been
      changed dynamically";
  document.body style.backgroundColor =
                        "lightblue";
    }
  </script>
</body>
</html>
```

output

Welcome to MY webpage

click the button to change the webpage

[click me]

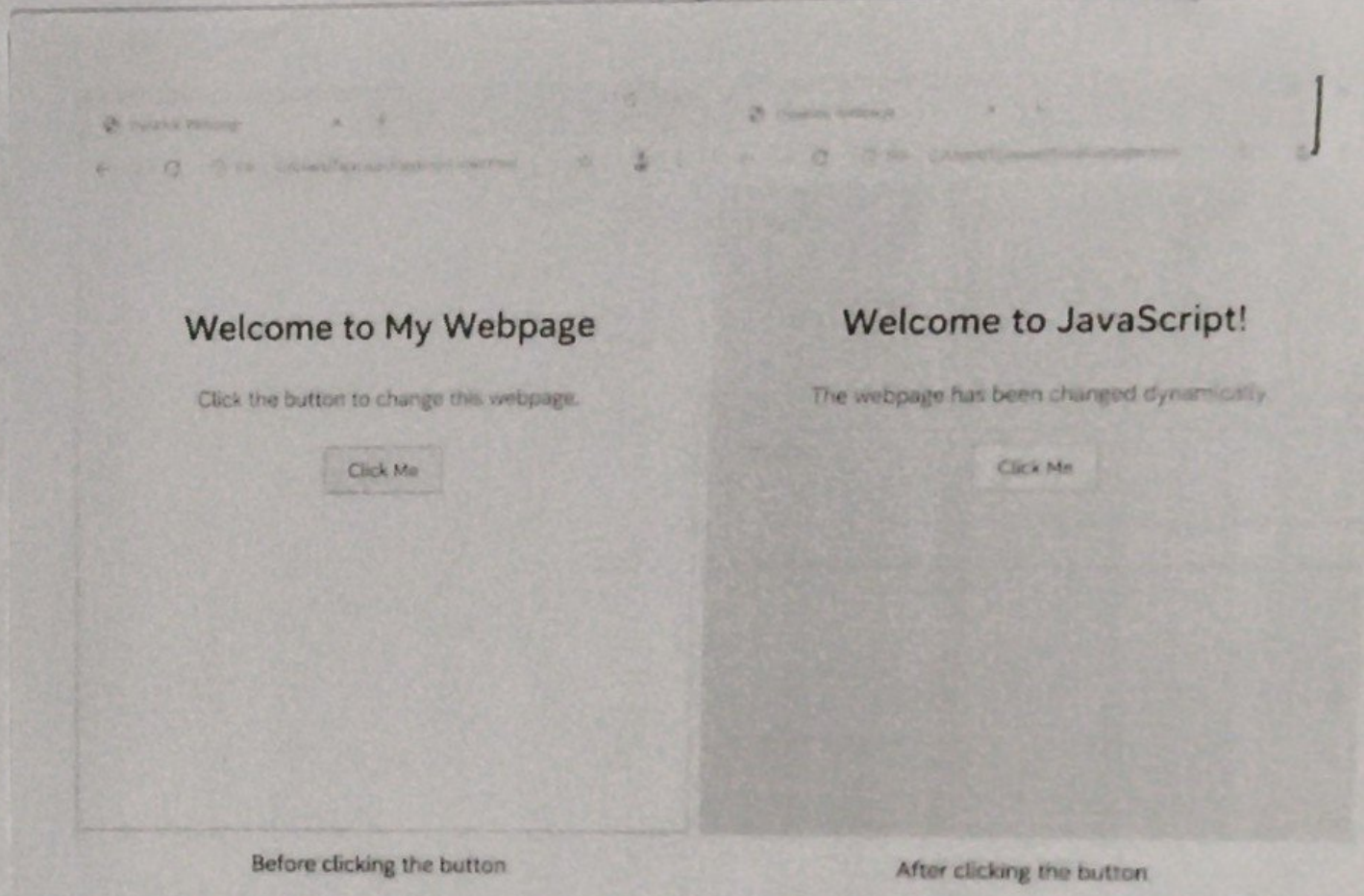
. when the user clicks "click me", the javascript function changepage() is called, then the heading dynamically changes and background also changes

After clicking the button:

welcome to Javascript!

The webpage has been changed dynamically

[click me]



Before clicking the button

After clicking the button

Experiment - 05

5] Implement Modules in Node JS

. On Node.js you can implement modules using 2 distinct systems : commonJS (JS), which is traditional default and ES modules (ESm), the modern javascript standard

1. the common JS method (default):

Common JS uses module.exports to share code & require() to import it this system load modules synchronously

a Create the module (math.js) :

```
const add = (a,b) => a+b ;
const subtract = (a,b) => a-b;
module.exports = {
        add ;
        subtract ;
}
```

b. use the module (app.js):

```
const math = require ( math);
console.log(math.add(10,5)).
console.log(math.subtract(10,5));
```

o/p: 15
5

2. the ES modules method (modern standard)

ES modules use export to share code and import to use it to make Node.js recognize i's modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a) create the module (calculator.js):

```
export const multiply = (a, b) => a * b
export const divide = (a, b) => a / b
const calculatorName = 'supercale';
export default calculatorName;
```

b) use the module (main.js)

```
import calculatorName, {multiply, divide}
    from './calculator.js';
console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

O/P:-

supercal
8
2

3. Using Built-in core modules:

Node.js also comes equipped with dozen of built-in core modules (like fs for file systems or path for file routing) that don't need installation. we will load them by module names:

```
// common JS style
const fs = require('fs');
// ES modules style
import fs from 'fs';
```

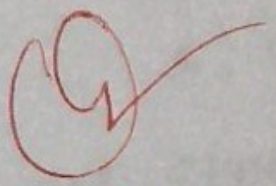
Here are some of the most commonly used ones;

1. fs = file system operations
2. http - HTTP server and client
3. path - File path utilities
4. os - operating system utilities
5. events - event handling
6. util - utility functions
7. stream - stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL query string handling

Example : simple HTTP server

```
http.create server ((req, res) => {
    res.write head (200, {'content-type':
        'text/html'});
    res.end ("Hello world!");
});.listen (8080);
```

o/p hello world!



Experiment-06

a) Develop mini application using express and NodeJS simple mini application using node JS & Express JS

Student management system:

1. Add a student
2. view all students
3. delete a student
4. uses express js routes
5. Simple HTML interface

Project Structure:

```
student-app
  ├ server.js
  └ public
      └ index.html
```

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
    res.json({ message: "student added
                        successfully",
    student : student });
});
```

```
//delete
app.delete("/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  students = students.filter(student =>
    student.id !== id);

  res.json({
    message: "student deleted succesfully"
  });
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server running at
    http://localhost:${PORT}`);
});
```

public/index.html :-

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management</title>
</head>
<body>
  <h1>Student Mangement System</h1>
<input type="text" id="name"
placeholder="Enter course">
<button onclick="addStudent()">
  Add Student</button>
```

```
<h2> Student List </h2>
<ul id="studentList"></ul>
<script>
    function addStudent(){
    let name = document.getElementById
                ("name").value;
    let course = document.getElementById
                ("course").value;
    fetch("/students", {
        method : "POST",
        headers : {
            "Content-Type":"application/json"},
        body : JSON.stringify({
            name : name,
            course : course
        })
    })
    .then(response => response.json())
    .then(data => {
                alert(data.message);
document.getElementById("name").value="";
document.getElementById("course").value="";
        displayStudents();
    });
}
```

```
function displayStudents() {
    fetch("/students").then(response =>
        response.json())
.then(students => {
    let list = document.getElementById
        ("studentList");
    let.innerHTML = "";
    students.forEach(student => {let li =
        document.createElement("li");
        li.innerHTML = student.id + " - " +
            student.name + " - " +
            student.course +
            "<button
onclick='deleteStudent(" + student.id +
")'>
Delete
</button>
        list.appendChild(li);
    });
});
}
function deleteStudent(id) {
    fetch("/students/" + id, {method:
        "DELETE"})
then(response => response.json())
then(data => {
    alert(data.message);
```

```
    displayStudents();
});

function displayStudents() {
</script>
</body>
</html>
```

output:-

Student Management System

Enter Student name

Enter course

Add student

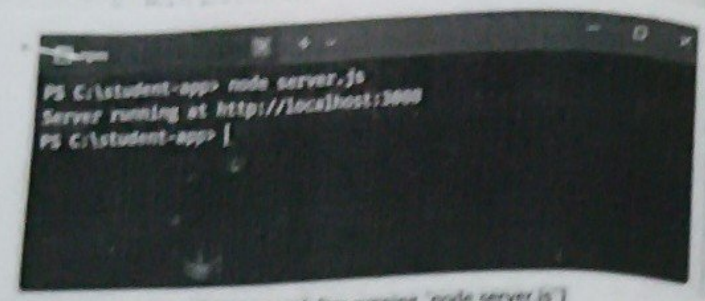
Student List:-

1. Monisha - CSM delete
2. Sita - CSE delete

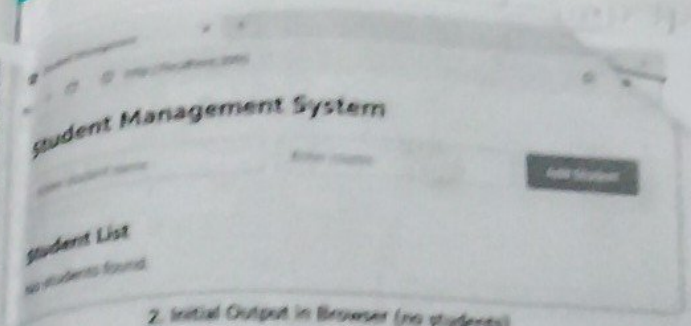
The Express.js server is successfully started at:

http://localhost:3000

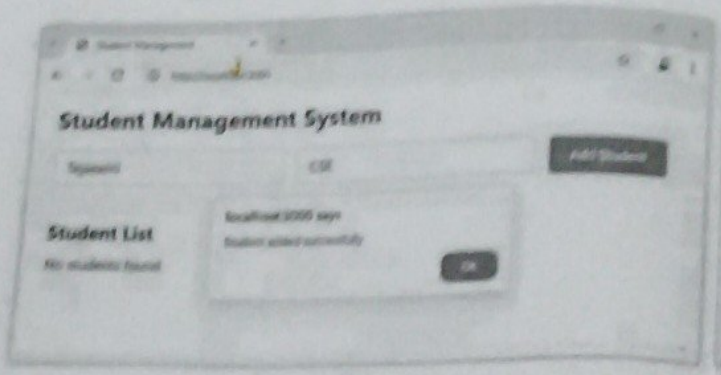
The browser displays the student management system with fields



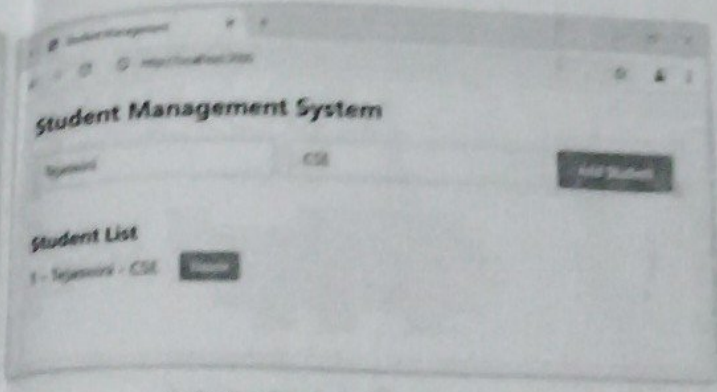
1. Terminal Output (after running `node server.js`)



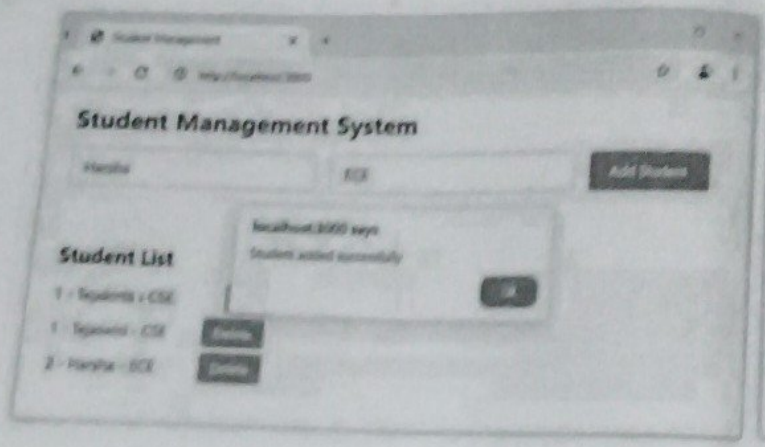
2. Initial Output in Browser (no students)



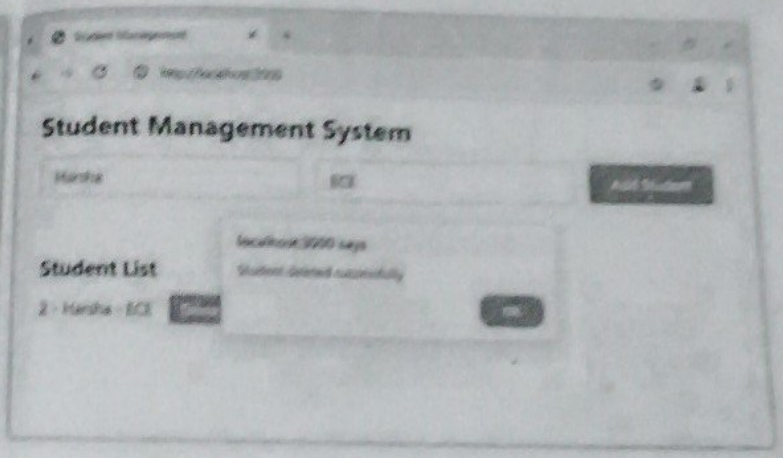
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-7

1) Create a functional component and class component using react

Aim: To develop a react application using functional components and class components to display student details.

Functional Component:-

In javascript function that returns JSX to display the user interface. It uses React hooks such as usestate to manage state and update the webpage dynamically

Program:-

```
import React, {useState} from
'react';
function studentFunctional(){
    const [name, setName] = useState("Monisha");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Teju");
    };
    const changeCourse() => {
        setCourse("CSD");
    };
    const incrementCount = () => {
        setCount(count+1);
    };
    return (
        <div style = {{
            border = "2px solid blue",
            padding : "20px",
            margin : "20px",
            borderRadius : "10px",
            textAlign : "center"
        }}>
        <h1> Student details </h1>
        <h2> Functional component </h2>
        <p> <b> Name : </b> {name} </p>
        <p> <b> Course : </b> {course} </p>
        <p> <b> Button clicks : </b> {count} </p>
        <button onClick = {changeName}>
            change Name </button>
        <button onClick = {changeCourse} change
            course </button>
        <button onClick = {incrementCount}> Increment
            </button>
        </div>
    );
```

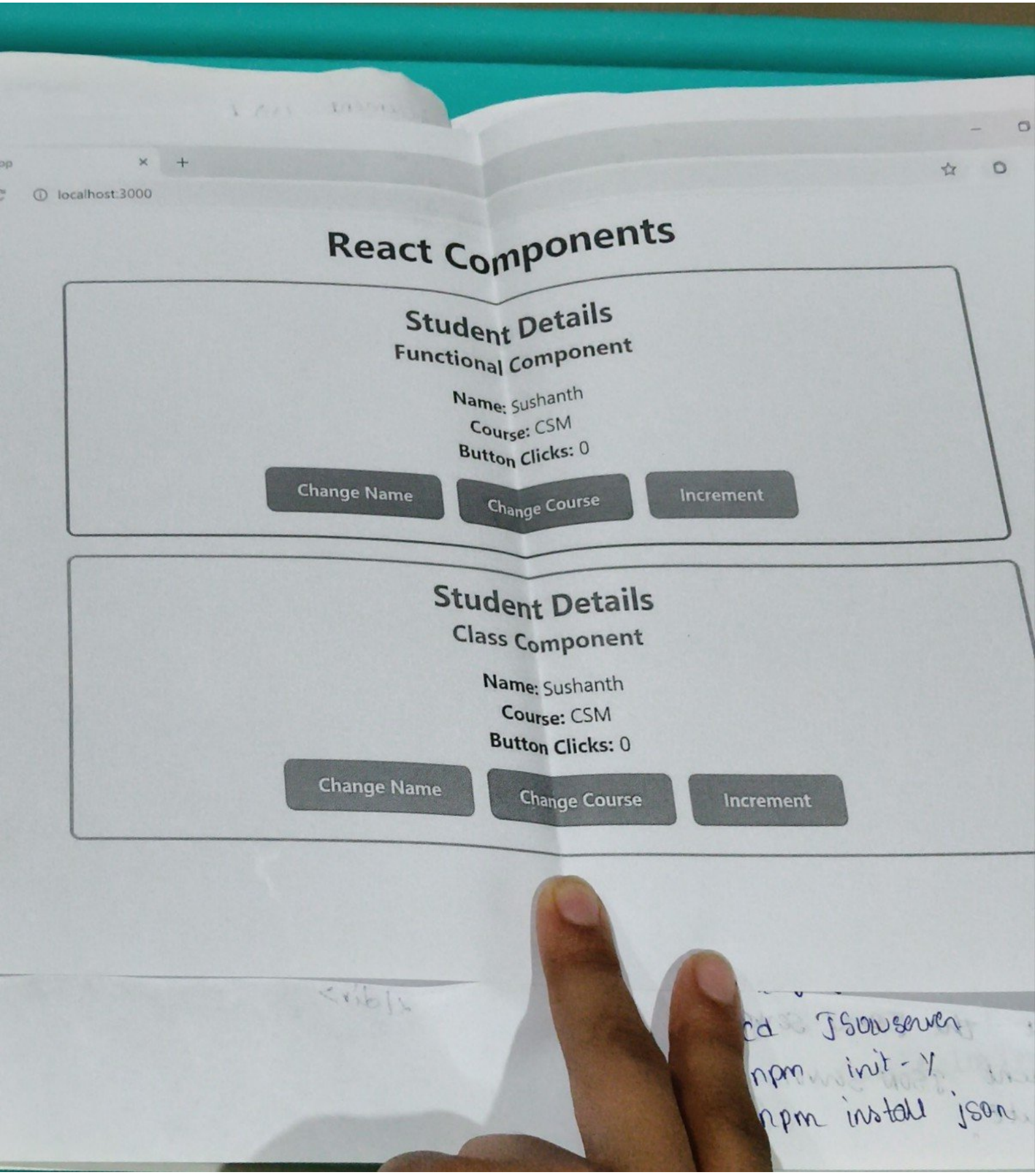
O/P:- when the user clicks the buttons, the name course & count change dynamically without reloding the webpage.

Class component:-

A class component is an ES6 javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state & this.setState().

Program:-

```
import React, {Component} from 'react';
class StudentClass extends Component {
    constructor(props) {
        super(props);
        this.state = {
            name = "Tejaswari",
            course = "computer science",
            count : 0
        };
    }
changeName = () => {
    this.setState({
        name: "Teju"
    });
};
changeCourse = () => {
    this.setState({
        course = "CSM"
    });
};
increment = () => {
    this.setState((prevState) => ({
        count: prevState.count + 1
    }));
};
render() {
    return (
        <div style = {{
            border : "2px solid green",
            padding : "30px",
            margin : "20px",
            borderRadius : "10px",
            textAlign : "center",
        }}>
        <h1> Student Details </h1>
        <h2> Class Component </h2>
    <p><b> Name : </b> {this.state.name}</p>
    <button onClick = {this.changeName}>
        changeName </button>
    <button onClick = {this.changeCourse}>
        change Course </button>
    <button onClick = {this.increment Count}>
        increment </button>
        </div>
```



cd JSONserver
npm init -y
npm install json

Experiment - 8

8] Implement http method using JSONServer

Aim:- TO implement http methods such as GET, POST, PUT, PATCH & DELETE using JSON server to perform CRUD (Create, Read, update & delete) operations on student records

Theory:- JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

GET - Retrieves data from the server
POST - Adds new data server
PUT - Replaces an existing record
PATCH - updates selected fields of a record
DELETE - Removes a record from the server

a) create a JSON server :-

Step 1:- Initalize the project

Open the vscode terminal & execute the following commands:-

```
mkdir JSONserver
cd JSONserver
npm init -y
npm install json server
```

Step2:- create db.json

create a file named db.json and add the following student data.

```
{
  "students": [
    {
      "id": "1",
      "name": "Tejaswani",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "chaitrika",
      "course": "CSM",
      "marks": 90
    },
    {"id": "3"
     "name": "sravani"
     "course": "IT",
     "marks": 88
}
```

Step 3:- Start the JSON server

For the current JSON server version, execute:

```
npx json-server db.json
```

The server start at:

http://localhost:3000

The student records can be accessed using:

http://localhost:3000/students

2. Implementing of HTTP methods:-

a. GET method:- The Get method is use retrive student records from JSON ser

Code:-

```
fetch('http://localhost:3000/studen
.then(response => response.json())
.then(data => {
      console.log(data);
})
.catch(error => {
      console.log(error);
});
```

o/p:-

b) POST METHOD:- The POST method is used to insert a new student record into the server

code:-

```
fetch ("http://localhost:3000/students",{
    method:'POST',
    headers:{
    'content-type':'application/json'
},
body: JSON.stringify({
    name: 'Rahul',
    course: 'CSE',
    marks: 92
})
})
.then(response => response.json())
.then(data =>{
    console.log(data);
})
.catch(error =>{
    console.log(error);
});
```

o/p

c) PUT METHOD:- The put method is used to replace an existing student record with updated data.

code:-

```
fetch ("http://localhost:3000/student/1",{
method: 'PUT',
headers:{
'content-type': 'application/json'
},
body: JSON.stringify({
    name: 'Tejaswini',
    course: 'AIML',
    marks: 95
```

```
    })
})
.then (response => response.json())
.then (data => {
    console.log (data);
})
.catch (error => {
    console.log(error);
});
```

O/P:-

D) Patch Method:- The PATCH method is used to update only selected fields of an existing student record.

code:- fetch ('http://localhost:3000/students/2', {
    method: 'PATCH',
    headers: {
        'Content-Type': 'application/json',
    },
    body: JSON.stringify ({
        marks: 96
    })
})
.then (response => response.json())
.then (data => {
    console.log (data)
})
.catch (error => {
    console.log (error);
});

O/P:-

E) DELETE METHOD:- The DELETE method is used to remove an existing student record from the server

code:- fetch ("http://localhost:3000/students/2", {
    method: 'DELETE'
})

```
body: JSON.stringify({
    marks: 96
})
})
.then(response => response.json())
.then(data => {
  console.log(data);
})
.catch(error => {
    console.log(error);
});
```

O/P:-

e) DELETE METHOD:- The DELETE method is used to remove an existing student record from the server

code:-

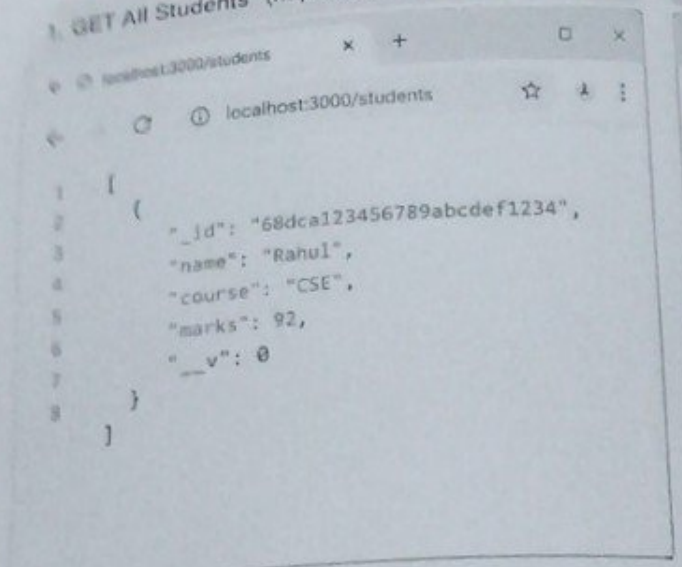
```
fetch("http://localhost:3000/stu
  {method: 'DELETE'
})
```

```
.then (response => {
    if (response.ok) {
        console.log("Student deleted
                    successfully");
    }
})
.catch (error => {
    console.log (error);
});
```

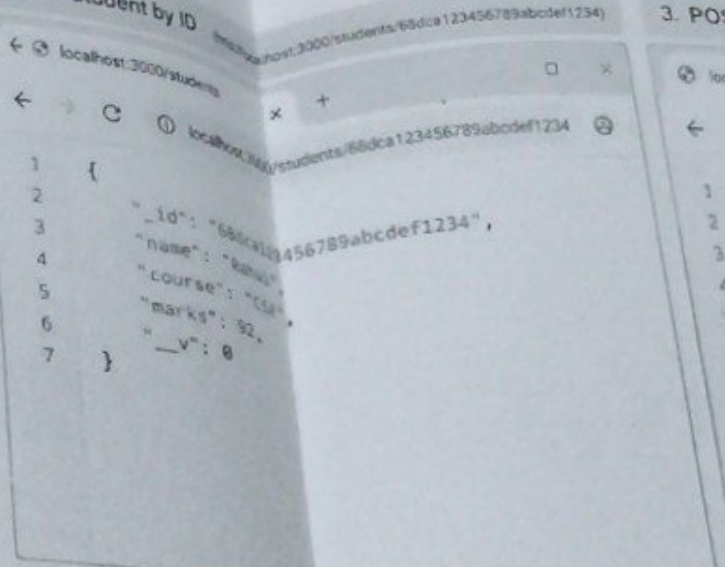
O/P:-

After executing the POST, PUT, PATCH & DELETE operations in sequence the remaining students will be:

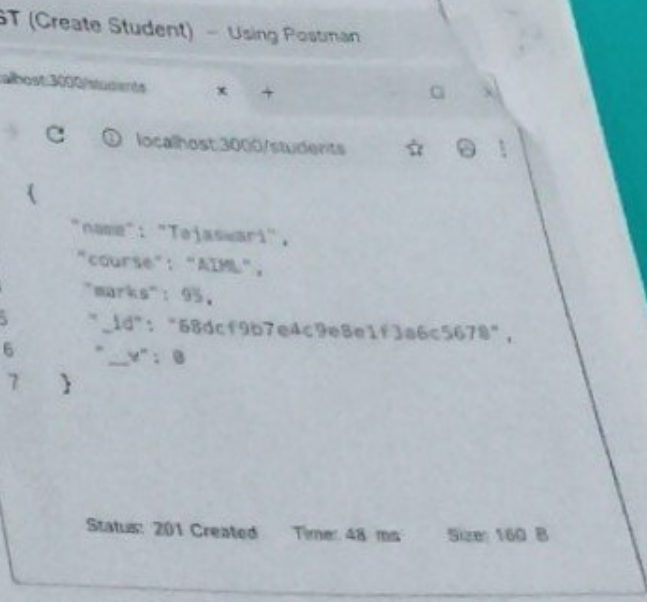
1. GET All Students (http://localhost:3000/students)



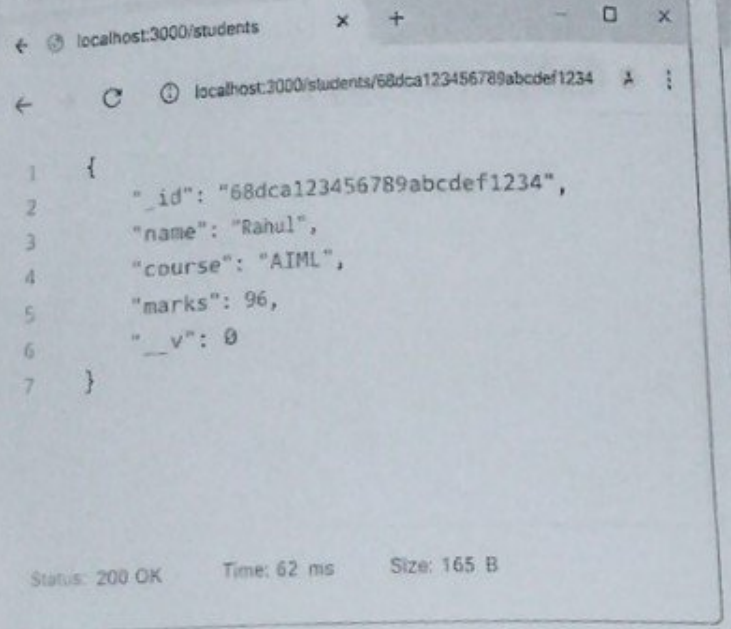
2. GET Student by ID



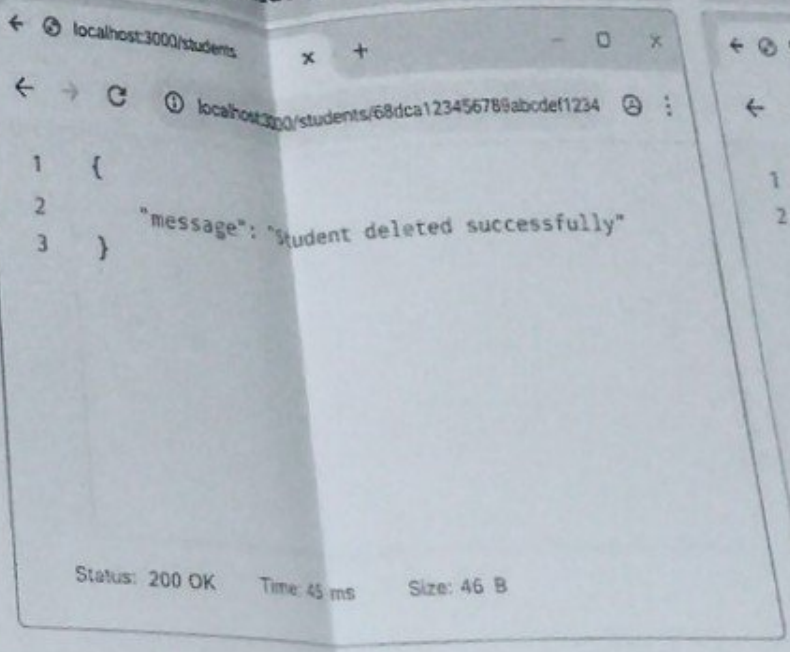
3. POST (Create Student) – Using Postman



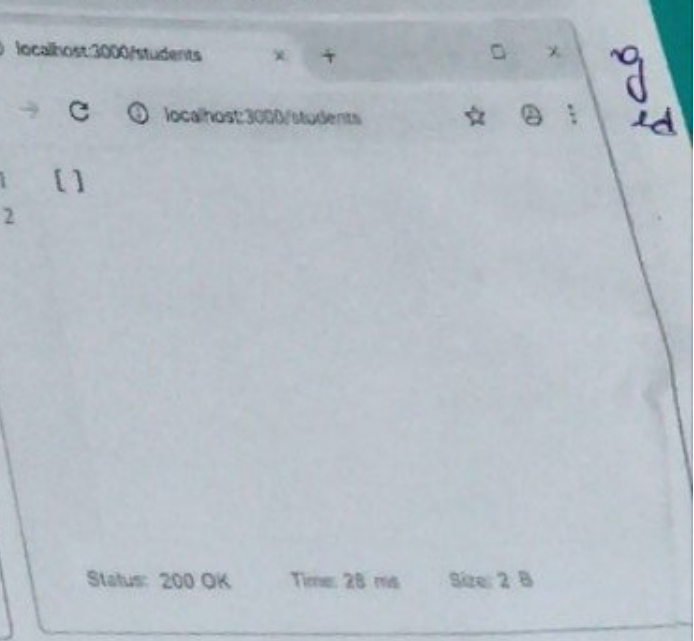
4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)



Experiment-9

9] Develop CRUD operations using MongoDB

Aim:- To develop a web application using Node.js, express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory:- MongoDB is a NoSQL database that stores data in flexible JSON like documents, CRUD operations are used to manage data in database.

Mongoose is an object Data modeling (ODM) library for MongoDB & Node.js It helps define schemas, interact with collections & perform database operations.

1. Installation & project setup:-

```
mkdir mongoDB-CRUD
cd mongoDB-CRUD
npm init -y
npm install express mongoose
```

Program :-

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/college')
.then(() => console.log('MongoDB Connected'))
.catch(err => console.log(err));
//Create Student
const StudentSchema = new
mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  course: {
    type: String,
    required: true
  },
  marks: {
    type: Number
    required: true
```

```
});

// Create Student model
const Student =
  mongoose.model('Student', Student
                            Schema);

// Create : Add
app.post('/students', async(req,res)=>{
  try{
    const student = new Student(req.
                                body);
const saved Student = await Student
                          save();
res.status(201).json(saved Student);
} catch(error){
    res.status(400).json({message:
      error.message});
  }
});

// READ : Get all Students
app.get('/students', async(req,res)=>{
  try{
    const Students = await
    Student.find();
    res.status(500).json({message:
      error.message});
  }
});

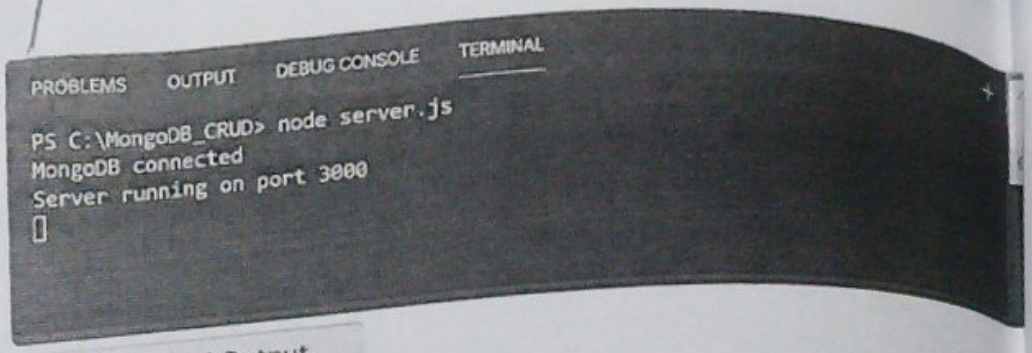
// Read : Get student by ID
app.get('/students/:id', async(req,res)=>{
  try{
    const student = await
    Student.findById(req.params.id);
    if(!student){
      return res.status(404).json({
        message: "Student not found"
      });
    }
    res.status(200).json(student);
} catch(error){
  req.status(400).json({message:
    error.message});
  }
});
```

// UPDATE

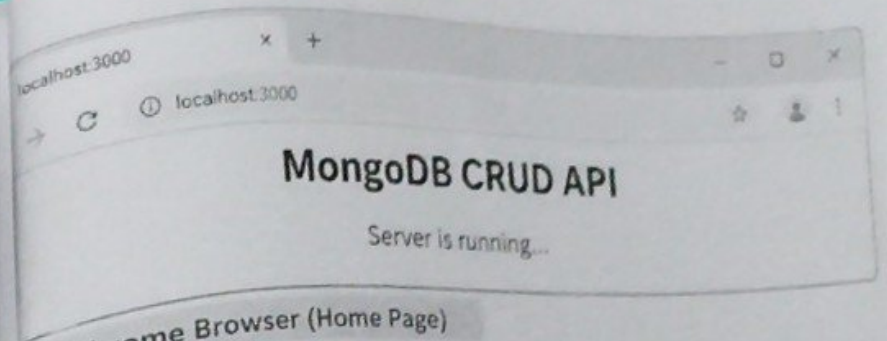
```
app.put('/students/:id', async (req, res) => {
  try {
    const student = await
    Student.findByIdAndUpdate(
      req.params.id,
      req.body
      {new: true, runValidators: true}
    );
if (!student) {
  return res.status(404).json({
    message: "Student not found"
  });
}
res.status(200).json(student);
} catch (error) {
  res.status(400).json({message:
    error.message});
}
```

```
try {
  const student = await Student.
      findByIdAndDelete(
        req.params.id
      );
  if (!student) {
    return res.status(404).json({
      message: "Student not found"
    });
  }
  res.status(200).json({
    message: "Student deleted successfully"
  });
} catch (error) {
  res.status(400).json({message:
    error.message});
}
});
```

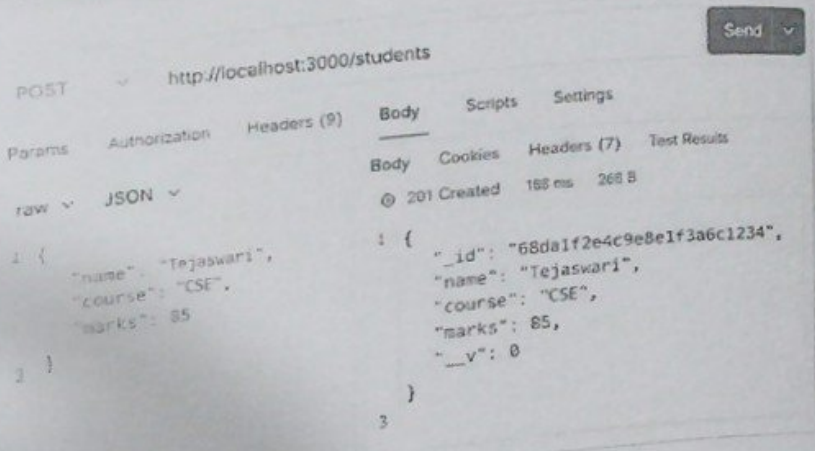
Op:-



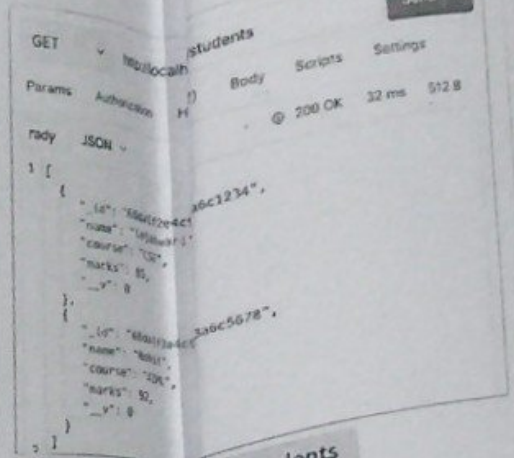
1. Terminal Output



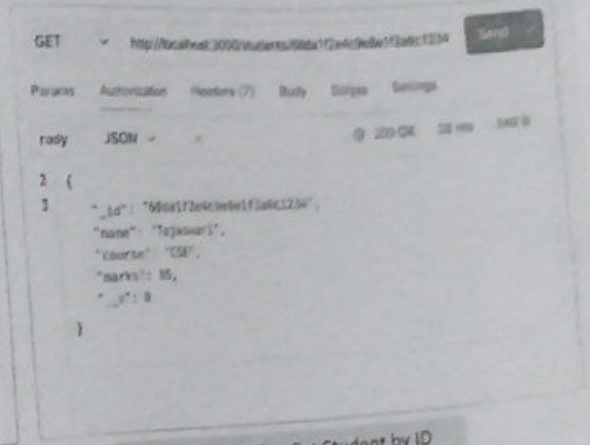
2. Chrome Browser (Home Page)



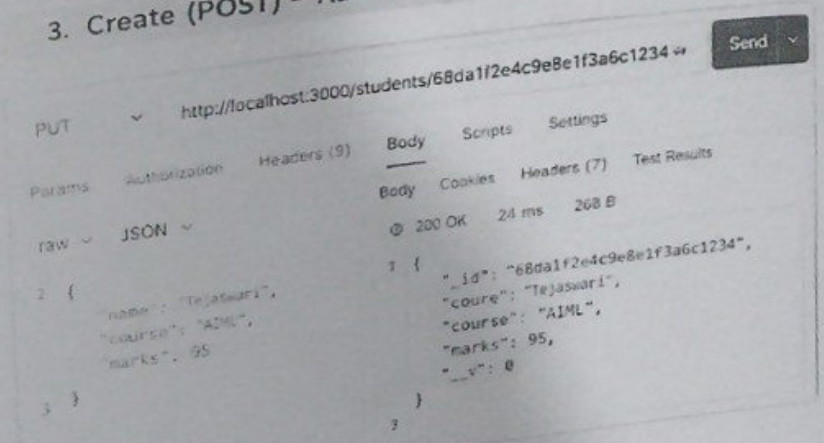
3. Create (POST) – Add Student



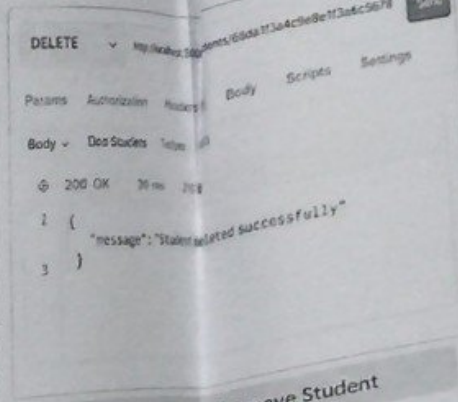
4. Read (GET) – All Students



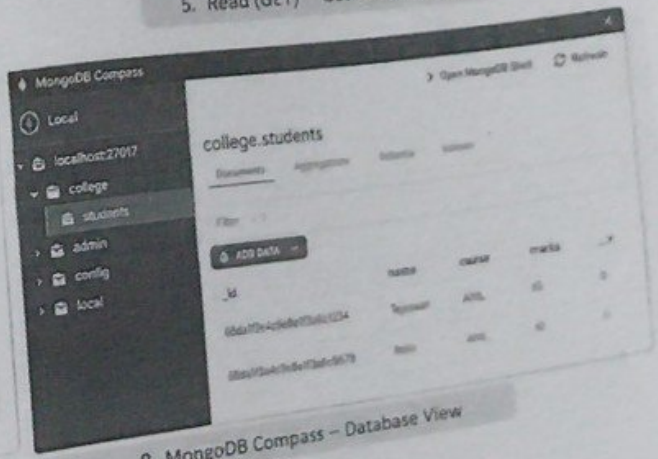
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View